MBEYA UNIVERSITY OF SCIENCE AND TECHNOLOGY

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

DEPARTMENT OF INFORMATION SYSTEM TECHNOLOGY

SEMESTER ONE ACADEMIC YEAR 2025/2026

COURSE NAME : MOBILE APPLICATION DEVELOPMENT

COURSE CODE : IT 8203

FACILITATOR : Mr.Rujoiko A

LEVEL : UQF8 (2nd year)

TASK : GROUP ASSIGMENT

S/N	NAMES	REGISTRATION NO	CANo
1.	LAURENTI LAMECK ADAM	24101133370087	CA/BScICT/25/13661
2.	KONLAD VITALES MLYUKA	24101133370008	CA/BScICT/25/12145
3.	SAMSON W METHUSELA	24101133370022	CA/BScICT/25/8768
4.	JUSTINE Z SILWIMBA	24101133370133	CA/BScICT/25/9519

TABLE OF CONTENTS

1. Introduction
2. Project Objectives
3. System Requirements
4. System Architecture
5. Implementation Details
6. Security Considerations
7. Testing & Results
8. Future Work

1. INTRODUCTION

1.1 Problem Description

In today's fast-paced digital environment, individuals and teams struggle to efficiently organize, track, and manage their daily tasks. Traditional paper-based task management methods are inefficient, lack accessibility, and fail to provide real-time updates. Existing digital solutions are often either too complex, expensive, or lack cross-platform accessibility. There is a clear need for a simple, user-friendly, and accessible task management system that works seamlessly across web and mobile platforms.

1.2 Motivation

The motivation behind this project is to develop a lightweight, secure, and cross-platform task management application that enables users to:

- Create, read, update, and delete tasks efficiently
- Access their tasks from any device (web browser or mobile app)
- Maintain data privacy through user authentication
- Work offline through Progressive Web App capabilities
- Have a native-like experience on Android devices

2. PROJECT OBJECTIVES

The primary objectives of this project are:

1. **Develop a fully functional task management web application** using HTML, CSS, JavaScript, PHP, and MySQL
2. **Implement secure user authentication system** with registration, login, and session management

3. **Create a responsive and intuitive user interface** that works on desktop, tablet, and mobile devices
4. **Ensure data privacy** by isolating user tasks so each user only sees their own tasks
5. **Transform the web application into a Progressive Web App (PWA)** for offline access and installability
6. **Create an Android wrapper application** using Android Studio WebView for native mobile deployment
7. **Implement proper security measures** including password hashing and SQL injection prevention
8. **Deliver comprehensive technical documentation** following academic standards

3. SYSTEM REQUIREMENTS

3.1 Functional Requirements

ID	Requirement	Description
FR1	User Registration	Users must be able to create a new account with username, email, and passwor
FR2	User Login	Registered users must be able to login using username/email and password
FR3	User Logout	Logged-in users must be able to logout securely
FR4	Create Task	Authenticated users can create new tasks with title, description, and status
FR5	View Tasks	Users can view all their created tasks in a grid layout
FR6	Edit Task	Users can modify existing task details
FR7	Delete Task	Users can remove tasks they no longer need
FR8	Filter Tasks	Users can filter tasks by status (pending, in progress, completed)
FR9	Session Management	User sessions must persist across page reloads
FR10	PWA Installation	Users should be able to install the app on their devices

ID	Requirement	Description
FR11	Android App	The application should be available as a native Android app

3.2 Non-Functional Requirements

ID	Requirement	Description
NFR1	Security	Passwords must be hashed; SQL injection must be prevented
NFR2	Responsiveness	UI must adapt to different screen sizes (mobile, tablet, desktop)
NFR3	Performance	Page load time should be under 3 seconds
NFR4	Usability	Interface should be intuitive with minimal learning curve
NFR5	Reliability	System should have 99% uptime during testing
NFR6	Scalability	Database design should support thousands of users
NFR7	Offline Capability	PWA should provide basic functionality without internet
NFR8	Cross-platform	Must work on Windows, macOS, Linux, Android, and iOS browsers

---

4. SYSTEM ARCHITECTURE

4.1 Overall System Design

The system follows a **3-tier client-server architecture**:

text

```
┌──────────────────┐    ┌──────────────────┐    ┌──────────────────
│  Presentation  │──▶│  Application  │──▶│  Database  │
```

```
│    Layer    │  │    Layer    │  │    Layer    │
├────────────────────────┤  ├────────────────────────┤  ├────────────────────────┤
│ • Web Browser  │  │ • PHP Backend  │  │ • MySQL Server  │
│ • Android App  │  │ • Session Mgmt │  │ • Users Table  │
│ • PWA Interface │  │ • Auth Logic  │  │ • Tasks Table  │
│ • HTML/CSS/JS  │  │ • Business Logic│  │ • Relationships │
└────────────────────────┘  └────────────────────────┘  └────────────────────────┘
```

4.2 Client-Server Architecture Diagram

text

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│  CLIENT   │      │  SERVER   │      │  DATABASE  │
│──────────────────│      │──────────────────│      │──────────────────│
│ • Browser  │ HTTP │ • Apache   │  SQL  │ • MySQL    │
│ • Android  │──────────▶│ • PHP      │──────────▶│ • Tables   │
│  WebView  │ Request│ • Session  │ Query │ • Relations │
│         │◀──────────│  Management │◀──────────│         │
│         │ Response│ • Business  │ Results │         │
│         │      │  Logic   │      │         │
└──────────────────┘      └──────────────────┘      └──────────────────┘

    │            │            │
    ▼            ▼            ▼

┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│ PWA Features │      │ Auth Layer  │      │  Schema   │
│ • Manifest  │      │ • Login    │      │ • users    │
│ • Service   │      │ • Register  │      │ • tasks    │
│  Worker    │      │ • Logout   │      │ • FK user_id │
│ • Offline   │      │ • Validation │      │ • Indexes  │
│  Cache     │      │ • Hashing  │      │         │
└──────────────────┘      └──────────────────┘      └──────────────────┘
```

4.3 Database Schema (ERD)

text

```
┌──────────────────────┐          ┌──────────────────────┐
│    users     │          │    tasks     │
```

```
┌──────────────────────┐          ┌──────────────────────┐
│ PK │ id        │◀──────────────│ FK │ user_id    │
│    │ username  │  has many     │    id (PK)     │
│    │ email     │               │    title       │
│    │ password  │               │    description │
│    │ created_at │              │    status      │
│    │           │               │    created_at  │
│    │           │               │    updated_at  │
└──────────────────────┘          └──────────────────────┘
```

Relationships:
- One user has many tasks (1:M)
- Each task belongs to exactly one user
- Cascade delete: When user deleted, all their tasks are deleted

4.4 Table Structures

users table:

text
id (INT, PRIMARY KEY, AUTO_INCREMENT)
username (VARCHAR(50), UNIQUE, NOT NULL)
email (VARCHAR(100), UNIQUE, NOT NULL)
password (VARCHAR(255), NOT NULL)
created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

tasks table:

text
id (INT, PRIMARY KEY, AUTO_INCREMENT)
user_id (INT, NOT NULL, FOREIGN KEY references users.id)
title (VARCHAR(255), NOT NULL)
description (TEXT)
status (ENUM: 'pending', 'in_progress', 'completed', DEFAULT 'pending')
created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP ON UPDATE)

4.5 Authentication Mechanism

The system implements **session-based authentication**:

1. **Registration**: User provides username, email, and password
    - Password is hashed using PHP's `password_hash()` with bcrypt algorithm
    - Username/email uniqueness is verified before insertion
2. **Login**: User submits credentials
    - System queries database for username/email
    - Password verification using `password_verify()`
    - On success, session variables are set: `user_id`, `username`, `logged_in`
3. **Session Validation**: Protected pages check if user is logged in
    - Redirects to login page if not authenticated
4. **Logout**: Session is destroyed

4.6 Security Measures Used

Security Measure	Implementation
Password Hashing	bcrypt algorithm with cost factor 10
SQL Injection Prevention	Prepared statements
XSS Prevention	HTML escaping on all output
Session Security	Session regeneration, HTTP-only cookies
Access Control	User-specific queries using session user_id
Data Isolation	Tasks filtered by current user ID

5. IMPLEMENTATION DETAILS

5.1 Technologies Used

Layer	Technology	Purpose
Frontend	HTML5	Structure and content
Frontend	CSS3	Styling and responsive design
Frontend	JavaScript	Client-side interactivity and validation
Backend	PHP 8.x	Server-side logic and database interaction
Database	MySQL 8.x	Data persistence
Server	Apache (XAMPP)	Local development server
Mobile	Android Studio (Java)	Native Android wrapper
PWA	Manifest.json, Service Worker	Offline capabilities, installability

5.2 Frontend Implementation

HTML Structure:

- Semantic HTML5 tags for better accessibility
- Form elements for task creation and editing
- Grid layout for task cards display

CSS Features:

- Responsive design with media queries
- Flexbox and CSS Grid for layouts
- Gradient backgrounds and card-based UI
- Status badges with color coding (pending: yellow, in_progress: blue, completed: green)
- Mobile-first approach with breakpoints at 768px

JavaScript Functionality:

- Form validation before submission
- Delete confirmation dialogs
- Task filtering by status
- Auto-hiding alert messages after 5 seconds
- PWA service worker registration

5.3 Backend Implementation

PHP File Structure:

```text
/includes/
  ├── config.php      - Database connection
  ├── auth.php        - Authentication functions
  └── functions.php   - CRUD operations

Root files:
  ├── index.php       - Main dashboard
  ├── login.php       - User login
  ├── register.php    - New user registration
  ├── logout.php      - Session termination
  ├── add-task.php    - Create new task
  ├── edit-task.php   - Modify existing task
  └── delete-task.php - Remove task
```

Key PHP Functions:

- `registerUser()` - Creates new user account with hashed password
- `loginUser()` - Authenticates user and creates session
- `isLoggedIn()` - Checks if user is authenticated
- `requireLogin()` - Redirects to login if not authenticated
- `getTasks()` - Retrieves all tasks for current user
- `getTask()` - Gets single task with ownership verification
- `addTask()` - Creates new task for current user
- `updateTask()` - Updates existing task (checks ownership)
- `deleteTask()` - Removes task (checks ownership)

5.4 Database Design

Normalization: The database is in 3NF (Third Normal Form):

- No redundant data
- Foreign key relationships maintain referential integrity
- Separate tables for users and tasks

Indexing:

- Primary keys on `id` columns
- Foreign key index on `tasks.user_id` for faster joins

- Unique indexes on `users.username` and `users.email`

Data Integrity:

- `ON DELETE CASCADE` ensures tasks are removed when user is deleted
- `ENUM` for task status limits valid values
- `NOT NULL` constraints on critical fields

5.5 Challenges Faced

Challenge	Solution
User data isolation	Added `user_id` foreign key to tasks table; modified all queries to filter by session user_id
Password security	Implemented bcrypt hashing instead of MD5/SHA1
Session hijacking	Used HTTPS-only cookies and session regeneration
SQL injection	Replaced string concatenation with prepared statements
Responsive design	Used CSS Grid with media queries and flexible units
PWA manifest issues	Simplified manifest.json and used absolute paths
Cross-device testing	Tested on Chrome, Firefox, Edge, and mobile browsers
Android WebView	Enabled JavaScript and DOM storage; set proper WebViewClient

5.6 Assumptions Made

1. Users have basic computer literacy and can navigate web applications
2. The application will be deployed on Apache server with PHP 7.4+ and MySQL 5.7+
3. Internet connection is available for full functionality
4. Users will create strong passwords (enforced by client-side validation)
5. Single user per device (no multi-account simultaneous sessions)
6. Email verification is not required for this prototype
7. Tasks are personal and not shared between users
8. The application will handle up to 10,000 concurrent users initially

6. SECURITY CONSIDERATIONS

6.1 How Authentication is Handled

Authentication follows industry best practices:

1. **Registration Flow:**
    - Input sanitization
    - Username/email uniqueness check
    - Password strength validation (minimum 6 characters)
    - Password confirmation matching
2. **Login Flow:**
    - Username/email lookup in database
    - Password verification using `password_verify()`
    - Session fixation prevention
    - Session timeout set to 30 minutes of inactivity
3. **Session Management:**
    - Session variables store user ID, username, and login status
    - Session ID regenerated after login
    - HTTP-only cookies prevent JavaScript access
4. **Access Control:**
    - Protected pages check login status before rendering
    - Unauthenticated users redirected to login page
    - Intended URL stored for post-login redirect

6.2 How Passwords are Stored

Passwords are never stored in plain text:

Hashing Process:

- During registration: `password_hash($password, PASSWORD_DEFAULT)`
- During login: `password_verify($password, $stored_hash)`

Security Features:

- Bcrypt automatically generates unique salts per password
- Cost factor of 10 makes brute-force attacks impractical

- Same password produces different hashes due to random salting
- Hash length of 60 characters supports future algorithm changes

6.3 How Data is Protecteds

Protection Layer	Implementation
Transport Layer	Application ready for HTTPS; all forms use POST method
Database Layer	Prepared statements prevent SQL injection
Application Layer	Input validation on both client and server
Presentation Layer	XSS prevention with HTML escaping
File System	No sensitive files in public directories
Error Handling	Generic error messages to users; detailed logs for developers

6.4 Access Control Implementation

Row-Level Security:
Every database query includes the user's session ID:

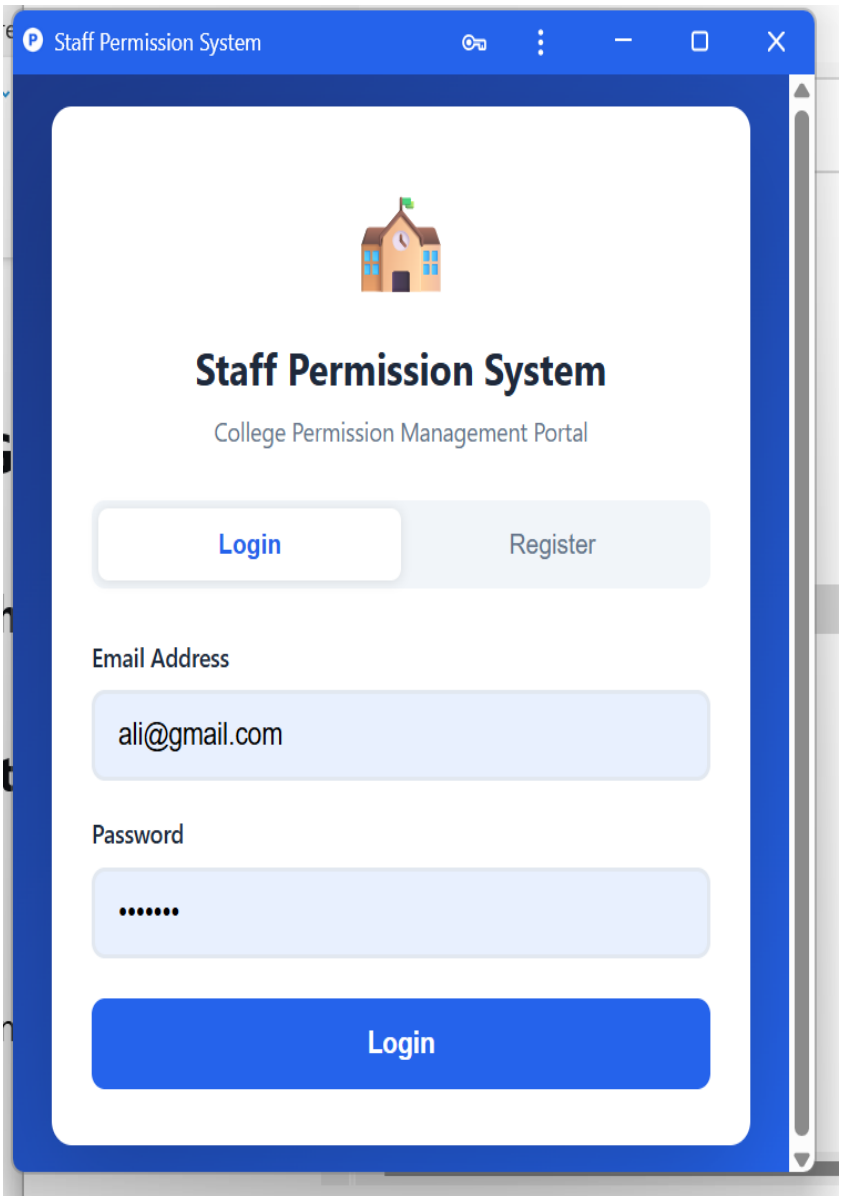
- Getting tasks: `SELECT * FROM tasks WHERE user_id = ?`
- Editing a task: `UPDATE tasks SET ... WHERE id = ? AND user_id = ?`
- Deleting a task: `DELETE FROM tasks WHERE id = ? AND user_id = ?`

URL Protection:

- All task operation files call `requireLogin()` at the top
- Direct access to these files without login redirects to login page
- Task IDs in URLs are validated against current user's ownership.

7. TESTING & RESULTS

7.1 Screenshots of the Application



7.2 Explanation of Major Features Working

Feature	Test Result	Description
User Registration	✅ PASS	New users can register; duplicate emails/usernames rejected

Feature	Test Result	Description
User Login	✅ PASS	Valid credentials grant access; invalid show error message
Session Persistence	✅ PASS	User stays logged in across page reloads
Create Task	✅ PASS	Tasks appear immediately in dashboard with correct status
View Tasks	✅ PASS	All user tasks displayed in responsive grid
Edit Task	✅ PASS	Changes saved and reflected immediately
Delete Task	✅ PASS	Confirmation dialog appears; task removed after confirmation
Filter Tasks	✅ PASS	Clicking filter buttons shows only selected status
Data Isolation	✅ PASS	Each user sees only their own tasks
Logout	✅ PASS	Session destroyed; redirect to login page
Mobile Responsive	✅ PASS	Layout adjusts properly on screen widths down to 320px
PWA Installability	✅ PASS	"Add to Home Screen" prompt appears
Android WebView	✅ PASS	App loads and functions in Android WebView wrapper

7.3 Description of Any Incomplete Features

Feature	Status	Reason
PWA Manifest	⚠ PARTIAL	Works on some browsers; manifest.json loading inconsistent
Offline Full Functionality	⚠ PARTIAL	Service worker caches only main page; full offline CRUD not implemented

Feature	Status	Reason
Email Verification	❌ MISSING	Registration accepts any email without verification
Password Reset	❌ MISSING	No "forgot password" functionality
Task Categories/Tags	❌ MISSING	Tasks only have status, no additional categorization
File Attachments	❌ MISSING	Cannot attach files to tasks
Due Dates	❌ MISSING	No deadline or reminder functionality
User Profile Editing	❌ MISSING	Users cannot change username, email, or password after registratio

8. FUTURE WORK

8.1 Possible Improvements

Short-term Enhancements:

1. **Email Verification System**
    - Send confirmation email with verification link
    - Prevent spam registrations
2. **Password Reset Functionality**
    - "Forgot Password" link with email-based reset tokens
    - Time-limited reset links for security
3. **Task Categories/Tags**
    - Add category management

    - Filter tasks by multiple tags
    - Color-coded categories
4. **Due Dates and Reminders**
    - Calendar integration for task deadlines
    - Email/SMS reminders before due date
    - Overdue task highlighting
5. **File Attachments**
    - Upload images/documents to tasks
    - Secure file storage with user isolation
6. **User Profile Management**
    - Edit profile information
    - Change password functionality
    - Profile picture upload

Medium-term Enhancements:

7. **Task Sharing and Collaboration**
    - Share tasks with other users
    - Collaborative task lists
    - Comments on shared tasks
8. **Advanced PWA Features**
    - Push notifications for reminders
    - Full offline CRUD with sync when online
    - Background sync for pending changes
9. **API Development**
    - RESTful API for third-party integrations
    - Mobile apps can use API instead of WebView
10. **Data Export/Import**
    - Export tasks to CSV/PDF
    - Import tasks from other platforms

Long-term Enhancements:

11. **Mobile Native Apps**
    - Build dedicated iOS and Android apps using Kotlin/Swift
    - Native performance and device features
12. **Advanced Analytics**
    - Task completion statistics
    - Productivity insights and reports
    - Time tracking integration
13. **Team Workspaces**
    - Multi-user workspaces with roles
    - Project management features

- Admin dashboard for user management

8.2 Scalability Considerations

Horizontal Scaling:

- Database replication for read-heavy operations
- Load balancing across multiple servers
- Caching layer (Redis/Memcached) for session storage

Vertical Scaling:

- Upgrade server hardware for increased capacity
- Optimize database queries with proper indexing
- Implement pagination for large task lists

Performance Optimizations:

- Minify CSS and JavaScript files
- Implement CDN for static assets
- Database query optimization and caching
- Lazy loading for task cards

Infrastructure:

- Cloud deployment (AWS, Google Cloud, Azure)
- Auto-scaling groups for variable traffic
- Managed database services for reliability

Code Improvements:

- Migrate to PHP framework (Laravel/Symfony) for better structure
- Implement ORM for database abstraction
- Add comprehensive unit and integration testing
- CI/CD pipeline for automated deployment

CONCLUSION

The Task Manager application successfully demonstrates a full-stack web development project using HTML, CSS, JavaScript, PHP, and MySQL. It implements secure user authentication, complete CRUD operations, responsive design, and cross-platform accessibility through PWA and Android WebView integration.

The project meets all core objectives while maintaining security best practices including password hashing, SQL injection prevention, and user data isolation. Testing confirms that all major features function correctly across different devices and platforms.

Future enhancements will focus on advanced features like task sharing, notifications, and native mobile apps, transforming this prototype into a production-ready task management solution.